

Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems

Yilun Wang

The Chinese University of HongKong
Hong Kong SAR, China

Guangba Yu*

The Chinese University of HongKong
Hong Kong SAR, China

Haiyu Huang

The Chinese University of HongKong
Hong Kong SAR, China

Yujie Huang

The Chinese University of HongKong
Hong Kong SAR, China

Zirui Wang

The Chinese University of HongKong
Hong Kong SAR, China

Pengfei Chen

Sun Yat-sen University
Guangzhou City, China

Michael R. Lyu

The Chinese University of HongKong
Hong Kong SAR, China

Abstract

LLM agents are increasingly explored for automating root cause analysis (RCA) in cloud-native systems, creating a need to evaluate both diagnostic correctness and the quality of the supporting investigation. Existing static benchmarks offer repeatable inputs but limited system-facing interaction, while live testbeds expose realistic tools but hinder controlled comparison because incident evidence varies across runs; both paradigms focus primarily on final answers. To address these limitations, we present Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It comprises 754 runtime-verified cases across 57 fault types on two microservice workloads spanning application services and Kubernetes platform layers. Each fault is captured as a state snapshot and replayed through standard diagnostic interfaces, with outcome labels and diagnostic evidence graphs that enable matched comparisons and process-level analysis. Across ten LLM agents, the strongest Joint RCA Accuracy (JRA) reaches 0.76 on OnlineBoutique and 0.68 on TrainTicket, while the corresponding Evidence Closure Rates (ECR) are only 0.38 and 0.15. This outcome–process gap shows that final-answer correctness alone substantially overestimates agents’ ability to perform evidence-grounded diagnosis.

Keywords

Large Language Models, Cloud Computing, Root Cause Analysis, Benchmark, Agentic AI

1 Introduction

Cloud-native systems host critical online services, but diagnosing their failures remains difficult because observable symptoms can propagate across application and platform layers [7, 27, 53, 59]. Root-cause analysis is therefore a deployed-software debugging task that spans application code, deployment specifications, runtime configurations, service dependencies, and infrastructure states. LLM-based agents offer a new approach to automating this task by selecting diagnostic tools and targets, interpreting returned observations, refining hypotheses, and consulting operational knowledge [9, 11, 19, 44]. This shift changes the evaluation target from root-cause prediction over supplied telemetry to an interactive

investigation whose conclusion must be supported by acquired observations.

Existing RCA benchmarks do not jointly support the conditions needed to evaluate this interactive investigation. Static telemetry benchmarks provide fixed incident artifacts and reproducible inputs, but they do not test whether an agent can select diagnostic tools and targets based on observations returned by the system [37, 40, 61]. Live-environment benchmarks preserve system-facing interaction, but the same injected fault may differ in severity, duration, or affected scope across runs [8, 18, 42]. As a result, two agents evaluated in separate live runs may face different fault conditions, so their performance differences cannot be attributed solely to the agents. Across both paradigms, outcome-oriented metrics evaluate the final diagnosis but cannot determine whether the agent has collected observations that causally justify its conclusion. Consequently, existing evaluations only partially measure interactive evidence acquisition, run-to-run comparability, and diagnostic process quality.

To address these gaps, we present Cloud-OpsBench¹, an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It comprises 754 fault cases across 57 fault types that span application services and Kubernetes platform layers, with OnlineBoutique and TrainTicket serving as representative workloads. Each fault is realized and runtime-verified in a live Kubernetes testbed before its verified incident state and associated temporal telemetry are captured as a state snapshot. During evaluation, snapshot-backed diagnostic interfaces replay the captured responses, allowing different agents to investigate the same realized incident while independently choosing their tools and targets. Each case pairs a root-cause label with a milestone-based evidence graph, enabling evaluation of root-cause correctness, milestone coverage, evidence-order consistency, and investigation efficiency.

We evaluate ten LLM agents on Cloud-OpsBench. First, outcome correctness and process grounding diverge: the strongest Joint RCA Accuracy (JRA) reaches 0.76 on OnlineBoutique and 0.68 on TrainTicket, while the corresponding Evidence Closure Rates (ECR) are only 0.38 and 0.15. This outcome–process gap shows that final-answer correctness alone substantially overestimates agents’ ability

*Guangba Yu is the corresponding author.

¹<https://github.com/LLM4Ops/Cloud-OpsBench>

to perform evidence-grounded diagnosis. Second, operational experience reuse can improve agents, especially on OnlineBoutique: trajectory-as-ICL raises DeepSeek-V4-Flash from 0.76 to 0.83 JRA and from 0.38 to 0.57 ECR, and SOP prompting yields similar gains; effects on TrainTicket are smaller and mixed. Third, failed trajectories cluster into four recurring categories spanning evidence interpretation, investigation control, root-cause resolution, and tool interaction.

In summary, this paper makes the following contributions:

- **A Runtime-Verified Fault Benchmark.** We construct Cloud-OpsBench with 754 fault cases spanning 57 fault types across application services and Kubernetes platform layers on two microservice workloads. A knowledge-guided pipeline converts fault specifications into executable injections and retains each case only after live runtime verification.
- **Decoupled Fault Realization and Interactive Evaluation.** We separate live fault realization from agent evaluation by capturing the tool-observable evidence of each verified incident as a state snapshot. Snapshot-backed interfaces then replay the same incident under matched observations while allowing agents to choose diagnostic tools and targets independently.
- **Process-Grounded Diagnostic Evaluation.** We pair root-cause labels with milestone-based evidence graphs and define metrics for milestone coverage, evidence-order consistency, and investigation efficiency. This design scores whether an investigation establishes the facts needed to justify its conclusion, without requiring a single expert command sequence.
- **Experiments on LLM-based RCA Agents.** We evaluate ten LLM agents on Cloud-OpsBench and find a pronounced outcome-process gap: strong Joint RCA Accuracy can accompany much lower Evidence Closure Rate, showing that final-answer correctness alone substantially overestimates agents' ability to perform evidence-grounded diagnosis.

2 Motivation

2.1 Agentic RCA

Cloud-native root-cause analysis is a deployed-software debugging task. In Kubernetes, software behavior is determined jointly by application code, deployment specifications, runtime configurations, service dependencies, and infrastructure states. A failure reported at an entry service may therefore originate several components or system layers away. Diagnosis must connect the reported symptom to a concrete faulty component and causal mechanism across these software and operational artifacts. This paper focuses on this task in Kubernetes-based cloud-native systems.

Agentic RCA makes this debugging process interactive. Given a user query describing an observed failure, an agent decides which system objects, logs, configurations, and dependency relations to inspect through diagnostic tools, and revises its hypotheses as observations accumulate [34, 48]. Consequently, two agents may produce the same root-cause label while differing substantially in whether their conclusions are supported by sufficient and causally coherent evidence. A benchmark for agentic RCA must therefore evaluate both the diagnostic outcome and the evidence-acquisition process that supports it.

Agents may also consult operational knowledge, such as API semantics and troubleshooting guidance, when interpreting observations [11, 19, 24]. Such knowledge can be provided as a controlled experimental input, but any accepted diagnosis must still be grounded in case-specific observations returned by tools.

2.2 Evaluation Gaps

Existing benchmarks leave complementary aspects of this investigation unmeasured. Static benchmarks offer reproducible incident artifacts but do not evaluate interactive evidence collection. Live-environment benchmarks support such interaction but expose agents to varying runtime observations, complicating process-level comparison. Across both paradigms, outcome-oriented metrics cannot determine whether the collected evidence justifies the diagnosis. Figure 1 and Table 1 summarize these limitations.

2.2.1 Gap 1: Lack of Interactive Diagnosis in Static Benchmarks.

Static RCA benchmarks typically provide pre-collected logs, metrics, or traces as fixed inputs [10, 37, 40, 61]. This design supports reproducible analysis, and preprocessing or code-based filtering can reduce the amount of telemetry presented to a model. However, it does not provide the system-facing interaction used in operational diagnosis, where an agent selects a diagnostic tool, specifies its target, and receives an observation before deciding what to inspect next. Consequently, the benchmark evaluates reasoning over supplied evidence rather than the agent's ability to acquire evidence through an adaptive investigation. Even settings that allow code execution over offline artifacts still operate on a supplied file corpus rather than on system-facing diagnostic interfaces [47].

In a static formulation of the TrainTicket DBReadOnly case, the model receives a fixed corpus of logs and telemetry and searches it offline for evidence such as read-only write rejections. This setting evaluates evidence retrieval and interpretation within a predetermined corpus. It does not evaluate whether, starting only from a query reporting a payment failure, the agent knows to query the dependencies of `ts-payment-service`, select `ts-order-service` from the returned topology, and request its logs. The missing capability is therefore interactive evidence collection, in which the agent chooses what to query next based on observations returned by previous diagnostic actions.

2.2.2 Gap 2: Variability Across Live Runs.

Live-environment benchmarks deploy microservices and inject faults during evaluation, preserving system-facing interaction and realistic runtime behavior [8, 18, 42]. Even when the fault type, target, and injection parameters are held constant, the resulting fault manifestation may vary in severity, duration, and affected scope across runs. The realized impact is jointly determined by the fault injection and the system's transient runtime state. Concurrent workloads, resource availability, scheduling decisions, and in-flight retries can amplify or attenuate the observable failure.

Such variation is valuable for evaluating robustness under realistic operational uncertainty. However, it complicates controlled agent comparison. When two agents are evaluated in separate live runs, both the diagnostic policy and the realized fault manifestation may differ. Observed performance differences therefore cannot be

Table 1: Comparison of Cloud-OpsBench with existing AIOps benchmarks across the three evaluation gaps.

Benchmark	Paradigm	System-Facing Interaction (Gap 1)	Run-to-Run Consistency (Gap 2)	Process Evaluation (Gap 3)
Static Telemetry Benchmarks				
LogHub [61]	Static Artifact	✗	✓	✗
RCAEval [37]	Static Artifact	✗	✓	✗
TimeSeriesBench [40]	Static Artifact	✗	✓	✗
Nezha [52]	Static Artifact	✗	✓	✗
OpenRCA [47]	Static Artifact	✗ ¹	✓	✗
Dynamic Environment Benchmarks				
AIOPSArena [42]	Live Environment	✗	✗	✗
AIOPSLab [8]	Live Environment	✓	✗	✗
ITBench [18]	Live Environment	✓	✗	✗
Proposed Benchmark				
Cloud-OpsBench	State Snapshot	✓ ²	✓	✓

¹ Code execution over supplied offline artifacts is not counted as system-facing interaction because it does not query an operational interface.

² Cloud-OpsBench replays captured cluster observations through snapshot-backed diagnostic interfaces for supported tool queries.

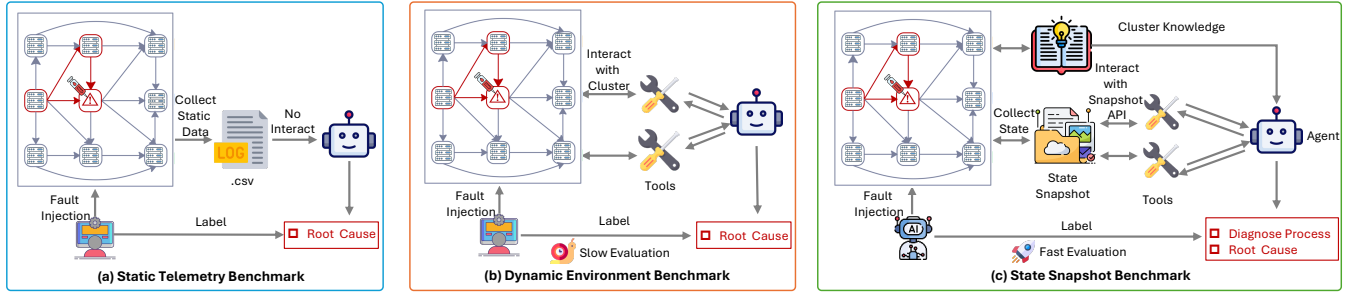


Figure 1: Benchmarking paradigms for agentic RCA. (a) Static telemetry: no system-facing interaction; outcome-only root-cause labels. (b) Live environments: interactive diagnosis, but still outcome-only evaluation, with run-to-run variability and high cost. (c) State snapshots (Cloud-OpsBench): matched interactive investigation over frozen incident evidence, with both process and outcome scoring.

attributed solely to the agents. Repeated trials can estimate average performance and run-to-run variance, but they do not provide matched conditions for controlled pairwise comparison.

In addition, live evaluation incurs substantial lifecycle overhead. Each independent run must establish a clean system state, warm up the workload, activate and verify the fault, allow the agent to complete its investigation, and restore the environment before the next trial. Because reliable estimates require repeated runs for every combination of fault case, agent, and experimental configuration, this lifecycle cost grows multiplicatively with benchmark scale. The resulting evaluation latency and infrastructure demand make large-scale comparison and rapid agent iteration difficult [55].

2.2.3 Gap 3: Outcome-Only Evaluation. Existing RCA benchmarks primarily evaluate whether the predicted component and fault type match the ground truth using metrics such as top- k accuracy, precision, recall, or F1-score [37, 42, 61]. These metrics remain necessary for measuring diagnostic correctness, but they provide little information about how an agent obtains its answer. Two agents receive identical outcome credit when they return the same correct diagnosis, even if one has collected observations that connect the reported symptom to the root cause while the other has produced an unsupported guess. Conversely, an agent that conducts a largely grounded investigation but makes an incorrect final attribution is indistinguishable from one that fails to investigate the system.

This creates a gap in the construct validity of agentic RCA evaluation. The claimed capability includes choosing diagnostic actions, acquiring relevant observations, and establishing a causal explanation, whereas outcome-only metrics measure only the final prediction. Evaluation should therefore complement diagnostic accuracy with process measures over the observable tool-mediated investigation rather than the agent’s self-reported reasoning [21, 45]. Such measures should assess the completeness, causal coherence, and efficiency of the investigation.

2.3 Benchmark Requirements

The preceding gaps yield three requirements for evaluating agentic RCA. **R1: System-facing evidence acquisition.** An agent must be able to select tools and targets through a defined set of standard diagnostic interfaces, receive captured system observations, and adapt subsequent actions to the returned evidence. **R2: Matched and reusable incident evidence.** Different agents must investigate the same realized incident under the same available evidence and deterministic query-response mapping without redeploying or re-injecting the fault for each evaluation. **R3: Tool-sequence-neutral process evaluation.** In addition to outcome correctness, the benchmark must assess the evidential completeness, causal order, and efficiency of the observable investigation without prescribing a canonical command sequence. Evidence-equivalent diagnostic facts and independent investigation orders should receive equal credit.

We scope evaluation to interactive investigation over preserved, runtime-verified incident evidence rather than to live remediation or open-ended environment mutation. These requirements guide the design of Cloud-OpsBench, introduced next.

3 Cloud-OpsBench

To satisfy these requirements, we present Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA comprising 754 fault cases across 57 fault types that span application services and Kubernetes platform layers. As illustrated in Fig. 2, Cloud-OpsBench realizes each fault in a live Kubernetes testbed through a knowledge-grounded construction pipeline (§ 3.2–3.3). Snapshot-backed diagnostic interfaces satisfy R1 by supporting adaptive investigation through a defined diagnostic toolset (§ 3.1). They satisfy R2 by replaying the same available incident evidence and query-response mapping without per-agent fault realization. Each case carries a milestone-based evidence graph that satisfies R3 by evaluating established diagnostic facts and their dependencies independently of the command sequence (§ 3.3.3).

3.1 Design Philosophy and Task Formulation

To bridge the tension between ecological validity and experimental control, we introduce the **State Snapshot** paradigm (Fig. 1(c)). Instead of evaluating agents directly against a live cluster, Cloud-OpsBench first realizes each fault in a real Kubernetes environment and captures its tool-observable context after the Verifier confirms the task-defining manifestation. Each snapshot combines resource, configuration, topology, and component states collected after verification with timestamped events, logs, and metrics from the case-specific incident interval between pre-injection baseline confirmation and manifestation verification. It therefore preserves both the verified incident state and its associated temporal telemetry without attempting to reproduce the complete system history.

During evaluation, agents interact with a mocked operational interface that replays responses from this snapshot for standard diagnostic commands (e.g., `kubect1 get pods`). This design does not eliminate noise from the original incident evidence. Rather, it eliminates evaluation-time environmental nondeterminism: the snapshot preserves evidence produced by a real fault realization while fixing the observation space for agent comparisons.

Within this controlled replay environment, we formalize Agentic RCA as a trajectory-based decision process. For each evaluation case j , the outcome ground truth is $\mathcal{R}_j^* = \langle C_j^*, F_j^* \rangle$, where C_j^* denotes the faulty component and F_j^* denotes the fault type. Its process-level ground truth is represented by a diagnostic evidence graph $\mathcal{D}_j^*$, which specifies the intermediate diagnostic states required to justify $\mathcal{R}_j^*$ and is constructed in § 3.3.3. An evaluation episode is modeled as

$$\Phi : \langle U_j, S_j \rangle \rightarrow \langle \mathcal{T}_j, \hat{\mathcal{R}}_j \rangle. \quad (1)$$

The agent receives only a user query U_j describing the observed failure and access to a diagnostic interface backed by the frozen snapshot S_j . The snapshot itself is not exposed to the agent. At each step, the agent issues a concrete tool use t_i , including the tool name and arguments, and the interface returns an observation o_i replayed from S_j . The evaluation harness records the resulting trajectory $\mathcal{T}_j = [(t_1, o_1), \dots, (t_n, o_n)]$. At termination, the agent returns only

the predicted diagnosis $\hat{\mathcal{R}}_j = \langle \hat{C}_j, \hat{F}_j \rangle$. We evaluate $\hat{\mathcal{R}}_j$ against $\mathcal{R}_j^*$ for outcome accuracy and $\mathcal{T}_j$ against $\mathcal{D}_j^*$ for process quality.

3.2 Phase 1: Fault Knowledge Base Construction

Cloud-OpsBench builds a fault taxonomy that spans both upper-layer microservices and the underlying infrastructure, synthesized from three complementary sources: (1) official Kubernetes documentation [20] for specification-based misconfigurations (e.g., incorrect service selectors); (2) developer communities [22, 41] for empirical operational faults; and (3) academic literature [4, 8, 18] for canonical failure modes.

Since raw sources combine unstructured prose with code snippets, we use Gemini 3 Pro as a structured extraction tool via few-shot prompting. For each fault type, the extraction process produces a reusable fault-knowledge entry comprising semantic metadata and a parameterized reproduction template. The metadata records the fault name, causal mechanism, expected local failure state, and possible upper-layer manifestations. The template specifies abstract prerequisites, the fault-inducing mutation or perturbation, and its activation logic using typed variables rather than workload-specific resource identifiers. For example, a service-selector-mismatch template requires the selector of a Service to differ from the labels of its intended backend Pods while leaving the namespace, Service, Pods, label keys, and label values unbound.

Candidate fault-knowledge entries undergo two pre-instantiation checks. First, an LLM-based consistency check identifies incomplete, ill-formed, or technically inconsistent specifications and refines their structured descriptions. Second, reviewers with cloud-native operations experience assess the causal logic of each fault and the technical feasibility of its reproduction template. Runtime validation is then performed on every case-specific instantiation in Phase 2. A fault type is retained in the released taxonomy only after at least one instantiated case passes this validation. This separation prevents an extracted template from being treated as executable or valid before it is instantiated and tested in a real cluster. Ultimately, Cloud-OpsBench contains 57 specific root-cause types across 8 categories spanning the Kubernetes stack.

We further tag each fault category with a diagnostic difficulty label (Easy/Medium/Hard) that reflects how hard it is to localize the faulty component and confirm its mechanism from the reported symptom (Tab. 2). The label is assigned at construction time from symptom observability and diagnostic distance, and is shared by all cases of that category.

3.3 Phase 2: Automated Fault Case Generation

Traditional fault injection relies heavily on manual orchestration, which is labor-intensive and difficult to scale. As illustrated in Fig. 2, Phase 2 uses a three-agent closed loop to instantiate the fault-knowledge entries from Phase 1 into executable fault plans and verified benchmark cases.

Testbed. To materialize the knowledge base into interactive cases, we establish a high-fidelity Kubernetes (v1.31) testbed with 4 nodes on Huawei Cloud ECS. As the foundation of cloud-native infrastructure, Kubernetes offers not only a complex distributed control plane for fault injection but also a standardized corpus

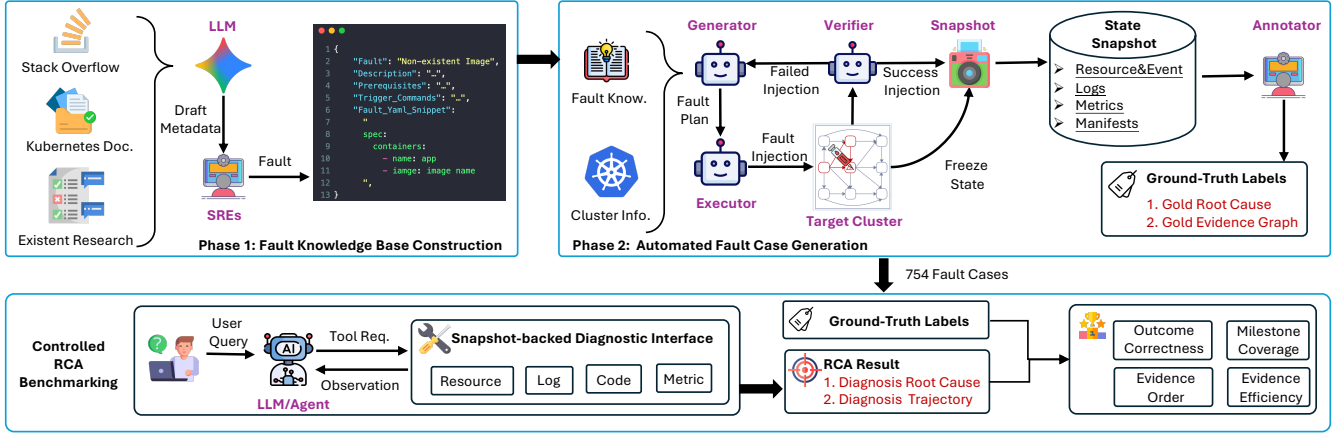


Figure 2: Overview of Cloud-OpsBench: Phase 1 builds the fault knowledge base; Phase 2 instantiates, verifies, and snapshots each case with process and outcome labels; agents then investigate through snapshot-backed diagnostic interfaces.

Table 2: Fault Taxonomy in Cloud-OpsBench. Easy/Medium/Hard correspond to the diagnostic reasoning complexity.

Fault Category (Mechanism Description)	Specific Fault Types (Full List)	Difficulty	# Cases
Admission Control Requests rejected by API server due to quota or permission violations.	NamespaceCPUQuotaExceeded, NamespaceMemoryQuotaExceeded, NamespacePodQuotaExceeded, NamespaceServiceQuotaExceeded, NamespaceStorageQuotaExceeded, MissingServiceAccount	Medium	58
Scheduling Pods stay Pending due to unsatisfied node constraints or affinity rules.	NodeCordonMismatch, NodeAffinityMismatch, NodeSelectorMismatch, PodAntiAffinityConflict, TaintTolerationMismatch, CPUCapacityMismatch, MemoryCapacityMismatch, PVBindingOccupied, PVCSelectorMismatch, PVCStorageClassMismatch, PVCCapacityMismatch, PVCAccessModeMismatch	Easy	164
Startup Pod creation fails.	VolumeMountPermissionDenied, MissingSecretBinding, IncorrectImageReference, ImageRegistryDNSFailure, MissingImagePullSecret	Easy	86
Runtime Application crashes or fails health probes during execution.	OOMKilled, LivenessProbeIncorrectProtocol, LivenessProbeIncorrectPort, LivenessProbeIncorrectTiming, ReadinessProbeIncorrectProtocol, ReadinessProbeIncorrectPort, ServiceSidecarPortConflict, MySQLInvalidCredentials, MySQLInvalidPort, DBReadOnly, DBConnectionExhaustion, DeploymentZeroReplicas	Easy	141
Service Routing Traffic routing failures between internal components.	ServiceSelectorMismatch, ServicePortMappingMismatch, ServiceProtocolMismatch, ServiceEnvVarAddressMismatch, GatewayMisroute, ServiceDNSResolutionFailure	Medium	91
Performance Performance degradation due to saturation or network drops.	PodCPUOverload, PodNetworkDelay, NodeNetworkDelay, NodeNetworkPacketLoss	Hard	76
Infrastructure Outages in underlying cluster control plane or components.	ContainerdUnavailable, KubeletUnavailable, KubeProxyUnavailable, KubeSchedulerUnavailable	Medium	40
Application Code Defect Application-level code defects in key business logic.	CodeMissingParameter, CodeBusyLoop, CodeExcessiveFileReads, CodeWrongReturn, CodeExcessiveFileWrites, CodeWrongArgumentOrder, CodeArtificialDelay, CodeMemoryLeak	Hard	98
Total	57 distinct fault types	-	754

of documentation. This allows us to rigorously test knowledge-grounded reasoning, distinguishing our approach from benchmarks based on proprietary or obscure systems. The observability plane integrates Prometheus [38] and Istio [17] for metrics, alongside native APIs for object states and logs. As target workloads, we deploy two widely adopted microservice benchmarks, **OnlineBoutique** (11 microservices) [14] and **TrainTicket** (41 microservices with database-backed services) [60], and generate user traffic with Locust [32]. We integrate ChaosBlade [6] for infrastructure-level perturbations such as network loss and disk pressure and use atomic kubectl commands for configuration mutations. Together, these mechanisms realize faults across application services and Kubernetes platform layers. For code-defect faults, we mutate service implementations with representative backend bug patterns, such as missing parameter values, wrong return values, or incorrect exception handling following [51]. Each mutation is packaged as a deployable service image for deployment.

3.3.1 Fault Injection Loop. For each candidate case j , the *Generator Agent*, implemented with Gemini 3 Pro, produces a case-specific fault plan $\mathcal{B}_j$ from a fault-knowledge entry and the target workload. Each plan specifies the required pre-injection state, resource selectors and operational parameters, ordered injection operations, and runtime validation assertions. The validation assertions cover mechanism activation, the expected local failure state, and any applicable upper-layer manifestation. Before each injection attempt, the *Executor Agent* restores the testbed to a clean deployment, resolves the selectors against the restored cluster state, establishes the required prerequisites, and confirms the pre-injection health baseline, including a successful end-to-end request for service-facing cases, before applying the current fault plan.

The *Verifier Agent* evaluates the validation assertions against the resulting cluster state using object states, events, logs, metrics, and connectivity results. It accepts an execution only when the fault-inducing condition is active at the target component, the expected

local failure state is present, and any specified upper-layer manifestation is observed. After successful verification, the resolved target and validated fault condition are recorded in the verified fault plan. This verification is necessary because Kubernetes recovery mechanisms may mask an injected fault. For example, a Pod may be rescheduled to another node and avoid a taint-induced scheduling failure. If validation fails, the Verifier compares the observed runtime state with the intended fault mechanism and returns structured feedback to the Generator, such as missing prerequisites, insufficient perturbation intensity, or an unintended recovery path. The Generator may then revise resource selectors or operational parameters while preserving the fault mechanism and validation criteria defined by the fault-knowledge entry. The clean deployment, injection, and verification procedure then repeats. Cases that still fail validation after at most three correction attempts are rejected rather than included in the benchmark.

A case is admitted to the snapshot-construction stage only after all runtime validation assertions are satisfied. After the Verifier confirms the observable manifestation used to define the task, we formulate U_j from that manifestation while excluding the faulty component, fault mechanism, and other construction-only information. The verified fault plan is retained as construction provenance but is neither exposed to the evaluated agent nor accepted as diagnostic evidence. Across the construction process, 800 candidate fault plans were generated, and 46 were rejected because they failed to manifest stable or diagnosable symptoms.

3.3.2 State Snapshots. After a fault case passes runtime verification, Cloud-OpsBench materializes its tool-observable context as a state snapshot (§ 3.1). The snapshot captures system state after verification and retains timestamped events, logs, and metrics from the corresponding case-specific incident interval. The snapshot builder records the baseline-confirmation and manifestation-verification timestamps and applies these boundaries when retrieving temporal evidence. To construct the replay interface, we pre-execute the diagnostic toolset (T1–T12, Table 3) against this verified incident context and store the resulting tool responses in a JSON repository. This repository serves as the persistence layer for deterministic replay. The snapshot-backed replay is therefore not a hand-written simulator. During evaluation, it returns the corresponding precomputed response. Before lookup, the interface validates each request against the supported tool schema and canonicalizes its critical arguments so that equivalent requests map to the same repository entry. A supported query targeting a non-existent object returns the captured failure response, such as `Not Found`. A valid request outside the enumerated replay coverage returns `UnsupportedQuery` and is not interpreted as evidence about the system state.

The snapshot builder covers four evidence channels. First, resource state and events are captured by executing retrieval and description tools (T1, T2) over Kubernetes objects in the target namespace. Second, application topology and telemetry are captured by retrieving static configurations (T3), dependency graphs (T4), logs (T5, T9), and reachability results (T6). Third, cluster-level context is captured by collecting node and component health information (T7, T10) and converting metrics into structured alerts (T8). Fourth, source-code evidence is captured by listing available code files and retrieving source code for application-level faults (T11,

T12). These four channels cover the standard evidence sources used in Kubernetes troubleshooting.

This design provides faithful replay with an explicit coverage boundary. For supported queries, the interface returns outputs captured from a verified incident in a real cluster rather than responses generated by a manually encoded model of Kubernetes behavior. Coverage is finite and measurable because the construction sweep enumerates the supported tools over case-valid arguments derived from the objects, namespaces, service endpoints, nodes, and source files available in each verified case. Snapshots for OnlineBoutique and TrainTicket contain an average of 599 and 1,708 precomputed responses per case and occupy 3.46 and 8.75 MiB, respectively. A case is retained only when subsequent annotation validation confirms that all required diagnostic facts are observable through the snapshot-backed interface. The finite replay repository bounds the observable evidence, but agents must independently choose tools and arguments as the investigation unfolds rather than follow a prescribed query list or command sequence. Through this construction, state snapshots address Gap 1 and Gap 2: agents investigate via system-facing diagnostic tools rather than offline telemetry dumps, while different agents face the same runtime-verified incident evidence and deterministic query–response mapping.

3.3.3 Ground-Truth Annotation. For each verified case j , Cloud-OpsBench defines outcome and process labels, $\mathcal{G}_j^* = \langle \mathcal{R}_j^*, \mathcal{D}_j^* \rangle$. The outcome label $\mathcal{R}_j^* = \langle C_j^*, F_j^* \rangle$ identifies the faulty component and fault type and is derived from the runtime-verified fault plan $\mathcal{B}_j$. Because outcome correctness alone cannot distinguish an evidence-backed diagnosis from an unsupported guess, the process label specifies the diagnostic facts needed to justify $\mathcal{R}_j^*$.

Process annotation. Using a single gold tool sequence would couple the evaluation to an annotator-preferred workflow and penalize agents that establish the same diagnosis through alternative tool sequences. By scoring established diagnostic facts and their dependencies instead of a canonical command script, the process label satisfies R3 and addresses Gap 3. We therefore represent the process label as a diagnostic evidence graph $\mathcal{D}_j^* = \langle \mathcal{M}_j^*, \mathcal{K}_j^*, \prec_j^*, \mathcal{Q}_j^* \rangle$. Each milestone $m \in \mathcal{M}_j^*$ is an atomic diagnostic fact rather than a raw observation, tentative hypothesis, or conditional branch. A proposition is split only when its parts can be established independently or support different downstream dependencies. For example, “the PVC is Pending” is a milestone, whereas “inspect the PVC” is a tool action and is not annotated. The set $\mathcal{K}_j^*$ partitions $\mathcal{M}_j^*$ into milestone groups. A singleton group represents a mandatory diagnostic fact, whereas a multi-milestone group contains interchangeable facts that make the same diagnostic advance. A process justification is complete when every group contains at least one established milestone. The directed acyclic relation $\prec_j^*$ records immediate evidential prerequisites among milestone groups, while its transitive closure defines the required diagnostic order and leaves groups not connected by a directed path unordered. For example,

$$U_j \rightarrow M_1 \rightarrow (M_2 \vee M_3) \rightarrow M_4 \begin{cases} \nearrow M_5 \\ \searrow M_6 \end{cases} \rightarrow \mathcal{R}_j^*.$$

For this graph, $\mathcal{K}_j^*$ contains $\{M_1\}$, $\{M_2, M_3\}$, $\{M_4\}$, $\{M_5\}$, and $\{M_6\}$. Here, U_j is the unscored initial diagnostic query and $\mathcal{R}_j^*$ is the outcome label. The group $\{M_2, M_3\}$ is completed by either M_2 or

Table 3: Diagnostic tools exposed by Cloud-OpsBench.

Tool Name (Arguments)	Returned Evidence
T1. GetResources(type, namespace, name, flags)	Resource lists or selected objects with status, metadata, and extended attributes.
T2. DescribeResource(type, name, namespace)	Runtime details of a Kubernetes object, including state, conditions, and events.
T3. GetAppYAML(service_name)	Deployment configuration YAML for the target application service.
T4. GetServiceDependencies(service_name)	Service dependency graph rooted at the target service.
T5. GetRecentLogs(service_name, namespace)	Recent service logs for runtime error inspection.
T6. CheckConnectivity(namespace, svc_name, port)	TCP reachability result for the target service endpoint.
T7. GetClusterConfiguration()	Cluster-wide node information, including resources, labels, taints, and status.
T8. GetAlerts()	Metric anomaly alerts generated by threshold-based detectors.
T9. GetErrorLogs(service_name, namespace)	Summary of log lines matching error, exception, or failure patterns.
T10. CheckNodeServiceStatus(node_name, component_name)	Liveness status of node-level or control-plane components.
T11. ListCodeFiles(app_name)	Available source files for an application service with brief file descriptions.
T12. GetSourceCode(app_name, file_path)	Source code content for a selected application file.

M_3 and serves as the prerequisite of $\{M_4\}$. Both $\{M_5\}$ and $\{M_6\}$ depend on $\{M_4\}$, but no dependency is specified between them, so they may be completed in either order. The graph therefore represents alternative diagnostic facts and independent milestone orders without enumerating separate tool trajectories.

For each milestone m , $Q_j^*(m)$ contains the admissible evidence bundles for establishing m . Each atomic evidence clause q specifies a diagnostic tool, admissible arguments, and a predicate over its normalized response. An evidence bundle is satisfied when all of its clauses have been observed. A milestone is established when at least one of its admissible bundles is satisfied. For example, $Q_j^*(m) = \{\{q_1\}, \{q_2, q_3\}\}$ (i.e., $q_1 \vee (q_2 \wedge q_3)$). This means that m can be established either by q_1 alone or by collecting q_2 and q_3 together. Thus, $\mathcal{K}_j^*$ represents alternative diagnostic facts, whereas $Q_j^*(m)$ represents alternative evidence bundles for establishing one fact. The annotation therefore permits alternative diagnostic facts, evidence sources, and independent investigation orders without prescribing a canonical command sequence.

Annotations were constructed by three members of the benchmark team, each with at least three years of experience in cloud fault diagnosis. For each case, process annotation starts from the causal backbone established during runtime verification § 3.3.1, which links the activated fault mechanism to the symptom presented in U_j . Annotators reverse this backbone from the reported symptom toward the fault mechanism and retain only atomic, snapshot-observable facts necessary for diagnosis as milestones. They then partition the retained milestones into the groups in $\mathcal{K}_j^*$. Two milestones are placed in the same group only when either one independently satisfies the same downstream dependency under the same prerequisite groups. The dependencies among these groups determine $\prec_j^*$.

To construct $Q_j^*(m)$, we traverse the complete per-case tool-response repository produced by the snapshot builder over the supported tools and enumerated case-valid arguments. Every evidence clause refers to an entry in this repository and matches a normalized response property that establishes the diagnostic fact represented by m . A response that is sufficient on its own forms a singleton evidence bundle, while complementary responses are grouped into a joint bundle.

Validation. Each instantiated annotation is automatically replayed against its frozen snapshot to check graph structure, case-specific binding consistency, clause executability, and evidence

satisfiability. Two auditors with practical experience in Kubernetes operations and cloud fault diagnosis independently review each annotation for causal sufficiency, milestone atomicity and non-redundancy, dependency correctness, and path neutrality across alternatives. They inspect U_j , $\mathcal{R}_j^*$, $\mathcal{D}_j^*$, and the frozen snapshot, but remain blinded to the construction-only fault plan. Disagreements and proposed revisions are adjudicated by a third senior practitioner with expertise in cloud-native operations. Following independent audit and adjudication, 9.4% of the graphs were revised and replay-validated, while the remaining graphs were accepted without modification. The retained graphs contain, on average, 3.3 milestones and 2.7 admissible evidence bundles per milestone.

4 Evaluation

In this section, we evaluate LLM agents on Cloud-OpsBench to answer the following RQs:

- **RQ1 (Outcome Effectiveness):** How accurately do agents identify the faulty component and fault type?
- **RQ2 (Evidence Process):** Do agents collect required evidence and respect causal dependencies?
- **RQ3 (Experience Reuse):** Can RCA agents benefit from historical diagnostic experience?

Additionally, we analyze recurring failure modes in incorrect diagnoses to characterize where current agents break down.

4.1 Experimental Setup

Setup and Models. We implement a baseline agent framework based on the standard ReAct [49] paradigm, utilizing Pydantic [39] for strict tool parameter validation. This custom architecture natively ensures full-trace observability of the agent’s reasoning trajectories and tool invocations. To guarantee reproducibility, we set the temperature to 0, cap the maximum interaction steps at 20, and disable extended thinking modes by default. We evaluate ten models covering both **proprietary LLMs** and **open-weight LLMs**. The proprietary LLMs include GPT-5, Claude-Sonnet-4, Gemini-2.5-Pro, Qwen3.5-Plus and Gemini-2.5-Flash, spanning frontier and latency-optimized variants. The open-weight LLMs include DeepSeek-V4-Flash, Qwen3-235B-A22B, Qwen3.5-27B, Qwen3-14B, and Qwen3-8B, covering high-capacity and compact variants[56].

Diagnostic-difficulty strata. We report selected results stratified by the construction-time diagnostic difficulty labels defined in

Table 4: Performance comparison of LLM agents on outcome and process metrics. The best results in each metric are bold.

Model	OnlineBoutique									TrainTicket								
	Outcome				Process					Outcome				Process				
	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	RAR↓	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	RAR↓
GPT-5	0.86	0.69	0.68	0.64	0.55	0.21	0.64	5.56	0.00	0.70	0.71	0.68	0.61	0.53	0.15	0.64	7.03	0.00
Claude-Sonnet-4	0.71	0.56	0.52	0.61	0.54	0.44	0.44	6.04	0.00	0.60	0.62	0.59	0.61	0.52	0.15	0.52	7.42	0.00
Gemini-2.5-Pro	0.81	0.69	0.67	0.74	0.68	0.46	0.63	6.21	0.00	0.62	0.61	0.58	0.58	0.51	0.14	0.51	7.13	0.00
Qwen3.5-Plus	0.80	0.68	0.64	0.70	0.64	0.34	0.65	5.92	0.00	0.67	0.79	0.64	0.64	0.58	0.25	0.63	5.90	0.00
Qwen3-235B-A22B	0.61	0.46	0.40	0.56	0.48	0.34	0.36	7.03	0.01	0.42	0.23	0.21	0.53	0.50	0.19	0.34	8.70	0.03
DeepSeek-V4-Flash	0.88	0.77	0.76	0.74	0.64	0.38	0.62	6.45	0.00	0.69	0.71	0.63	0.62	0.56	0.24	0.53	7.68	0.00
Gemini-2.5-Flash	0.82	0.67	0.65	0.71	0.67	0.37	0.58	6.49	0.01	0.60	0.55	0.53	0.45	0.42	0.12	0.48	4.91	0.00
Qwen3.5-27B	0.85	0.71	0.70	0.75	0.70	0.43	0.54	6.87	0.01	0.62	0.64	0.58	0.59	0.55	0.18	0.55	7.05	0.01
Qwen3-14B	0.61	0.42	0.41	0.70	0.57	0.45	0.32	14.58	0.28	0.41	0.25	0.24	0.60	0.52	0.20	0.37	14.53	0.31
Qwen3-8B	0.35	0.19	0.19	0.58	0.40	0.16	0.37	16.01	0.46	0.38	0.26	0.22	0.53	0.45	0.06	0.45	11.55	0.22

§ 3.2 (Tab. 2). Each evaluation case inherits its category label; we do not reassign difficulty from agent scores. Within each system and difficulty band, we compute metrics per model and then average equally across the ten models.

4.1.1 Evaluation Metrics. We evaluate agents along three complementary dimensions: outcome correctness, process grounding, and diagnostic efficiency.

(1) Outcome Correctness. Our primary metric is **Joint RCA Accuracy (JRA)**, the fraction of episodes for which $\hat{\mathcal{R}}_j = \mathcal{R}_j^*$, equivalently, $\hat{C}_j = C_j^*$ and $\hat{F}_j = F_j^*$. We also report **Component Accuracy (CA)** and **Fault-Type Accuracy (FA)** to distinguish localization errors from fault-type classification errors. An episode with a missing or invalid field is counted as incorrect for the corresponding accuracy metric.

(2) Process Grounding. We evaluate process quality by projecting each trajectory $\mathcal{T}_j$ onto its diagnostic evidence graph $\mathcal{D}_j^*$ (§ 3.3.3), following process supervision [31] and milestone-based agent evaluation [33, 35]. We scan $\mathcal{T}_j$ chronologically and record every milestone whose evidence condition is satisfied to obtain the established milestone set $\mathcal{M}_j^{\text{est}}$. A milestone group is completed when at least one of its member milestones is established, giving $\mathcal{K}_j^{\text{est}} = \{k \in \mathcal{K}_j^* \mid k \cap \mathcal{M}_j^{\text{est}} \neq \emptyset\}$. During the same scan, a group k enters the order-grounded set $\mathcal{K}_j^{\text{ord}}$ when the current observation completes or refreshes an admissible evidence bundle for one of its milestones after every ancestor group in the reachability relation of $\prec_j^*$ is already in $\mathcal{K}_j^{\text{ord}}$. We define $\text{MC}_j = |\mathcal{K}_j^{\text{est}}|/|\mathcal{K}_j^*|$, $\text{EOC}_j = |\mathcal{K}_j^{\text{ord}}|/|\mathcal{K}_j^*|$. **Milestone Coverage (MC)** measures the fraction of milestone groups completed by the trajectory. **Evidence-Order Consistency (EOC)** measures the fraction of required milestone groups that are established in an admissible causal order. A missing group receives zero under both metrics and prevents its dependent groups from receiving EOC credit. We further define **Evidence Closure Rate (ECR)** as a binary completeness indicator: $\text{ECR}_j = 1 \left[\mathcal{K}_j^{\text{est}} = \mathcal{K}_j^* \right]$. ECR equals 1 only when the trajectory completes every required milestone group, and 0 otherwise. We report both metrics as macro averages over cases.

(3) Diagnostic Efficiency. **Evidence Efficiency (EE)** is the fraction of tool calls that supply clauses in an admissible evidence

bundle completed when a group first enters $\mathcal{K}_j^{\text{ord}}$. All calls supplying a completed joint bundle receive credit, while later evidence for an already grounded group receives no additional credit. We additionally report **Steps**, the total number of tool invocations, and **Redundant Action Rate (RAR)**, the fraction of calls that repeat an earlier tool-and-argument pair without contributing new evidence or advancing a new group. We interpret these efficiency metrics jointly with MC and EOC to avoid rewarding premature termination.

4.2 RQ1: RCA Outcome Effectiveness

Kubernetes-visible symptoms do not directly reveal root causes. Cloud-OpsBench exposes rich Kubernetes-level evidence, such as pod names, deployment states, events, service selectors, endpoints, and application logs. As a result, agents can often identify the affected object even when they fail to infer the underlying fault mechanism. This is especially clear on OnlineBoutique, where CA is consistently higher than FA and JRA. For example, DeepSeek-V4-Flash achieves 0.88 CA but only 0.77 FA and 0.76 JRA, while GPT-5 achieves 0.86 CA but drops to 0.69 FA and 0.68 JRA. The gap suggests that agents frequently locate the right Kubernetes component but confuse similar root-cause mechanisms, such as image reference errors vs. registry failures, or service selector, port, and protocol misconfigurations.

On TrainTicket, the relation between CA and FA is less monotonic because the system contains many homogeneous services connected through gateways and databases. Some faults expose strong type-level symptoms but obscure the responsible service through cross-service propagation, while others identify the affected service but leave the configuration-level cause ambiguous. For instance, GPT-5 obtains comparable CA and FA on TrainTicket (0.70 and 0.71), whereas Qwen3.5-Plus shows higher FA than CA (0.79 vs. 0.67), indicating that it often recognizes the fault class but does not always localize the responsible service. Therefore, Cloud-OpsBench distinguishes between object-level symptom localization and mechanism-level root-cause attribution, and JRA captures the stricter requirement that both must be correct.

Open-weight models can be competitive for cloud RCA. Strong RCA performance is not limited to the largest proprietary models. On OnlineBoutique, DeepSeek-V4-Flash achieves the best

outcome scores among the evaluated models, with 0.88 CA, 0.77 FA, and 0.76 JRA. Qwen3.5-27B is also competitive, reaching 0.85 CA, 0.71 FA, and 0.70 JRA, close to or better than GPT-5 (0.86 CA, 0.69 FA, and 0.68 JRA) and Gemini-2.5-Pro (0.81 CA, 0.69 FA, and 0.67 JRA). RCA agents inspect operational artifacts such as logs, resource manifests, service topology, and runtime events, which can be sensitive or expensive to process through remote proprietary APIs. Competitive open-weight models therefore provide additional deployment choices for RCA, especially when cost, latency, or data-governance constraints matter.

RCA difficulty is governed by symptom observability and diagnostic distance. Using the pre-defined diagnostic-difficulty strata (§ 3.2; Experimental Setup), as summarized in Tab. 2, Startup, Scheduling, and Runtime faults are labeled Easy (391 cases); Admission Control, Service Routing, and Infrastructure faults are labeled Medium (189 cases); and Performance and Application Code Defect faults are labeled Hard (174 cases). Fig. 3 visualizes model performance across Easy, Medium, and Hard faults using JRA and ECR. The JRA heatmaps show a clear difficulty gradient. Averaged across models, OnlineBoutique drops from 0.73 JRA on Easy faults to 0.51 on Medium and 0.30 on Hard faults; TrainTicket drops from 0.67 to 0.43 and further to 0.08. This confirms that the difficulty split captures meaningful differences in root-cause attribution rather than only surface-level task labels.

The three difficulty levels differ in symptom observability and diagnostic distance. Easy faults, such as startup, scheduling, and runtime failures, often expose direct Kubernetes-level symptoms: unhealthy pods, image-pull failures, unschedulable pods, probe failures, or explicit runtime errors. Medium faults, including admission, service, and infrastructure failures, require correlating multiple Kubernetes objects, such as deployments, services, endpoints, quotas, nodes, and control-plane components. Hard faults, including performance and code-defect cases, often still expose abnormal alerts, metrics, or logs, so the evidence is not completely hidden. However, these signals are indirect: they describe where system behavior changes, but not necessarily which component or mechanism caused the change.

Solving Hard faults therefore requires stronger telemetry attribution and source analysis capability, rather than merely observing abnormal signals. This explains why agents may still collect relevant evidence in Hard cases, as reflected by nonzero ECR, while achieving much lower JRA.

4.3 RQ2: RCA Process Alignment

Outcome correctness overstates diagnostic grounding. A correct final RCA does not necessarily imply that the diagnosis is supported by a complete evidence chain. In Cloud-OpsBench, JRA measures whether an agent correctly identifies both the faulty component and the fault type, while ECR measures whether the agent has closed the required diagnostic evidence graph. The gap between these two metrics shows that outcome-only evaluation can overestimate the reliability of RCA agents.

This pattern is consistent across systems and models. On OnlineBoutique, DeepSeek-V4-Flash achieves the strongest outcome performance with 0.76 JRA, but its ECR is only 0.38. GPT-5 shows an even larger gap, with 0.68 JRA but only 0.21 ECR. Qwen3.5-27B

similarly reaches 0.70 JRA while closing the evidence graph in only 0.43 of cases. On TrainTicket, GPT-5 obtains 0.68 JRA but 0.15 ECR, and DeepSeek-V4-Flash obtains 0.63 JRA but 0.24 ECR. These results indicate that agents can often infer the correct component-fault pair from strong local signals, such as abnormal pod status, alerts, or service errors, without establishing the full diagnostic chain that would make the conclusion explainable.

This finding shows a form of *ungrounded diagnosis*: the final RCA label may be correct, but the supporting evidence remains incomplete. For SREs, this distinction matters because a correct RCA label is not enough; the diagnosis should also show which evidence supports the intervention.

Agents perform broad but shallow evidence exploration. Agents are rarely completely off-path. Their MC scores are often moderate to high, suggesting that they can reach many expert-defined diagnostic milestones and inspect relevant Kubernetes objects, logs, or configuration artifacts. However, these partial discoveries do not usually accumulate into complete diagnostic closure. On OnlineBoutique, Qwen3.5-27B reaches 0.75 MC but only 0.43 ECR, Gemini-2.5-Pro reaches 0.74 MC but 0.46 ECR, and GPT-5 reaches 0.64 MC but only 0.21 ECR. On TrainTicket, Qwen3.5-Plus and DeepSeek-V4-Flash reach 0.64 and 0.62 MC, while their ECR remains 0.25 and 0.24, respectively. These gaps suggest that agents often touch the right evidence regions, but their investigations remain local, fragmented, and incomplete.

EOC introduces a smaller but consistent reduction from MC by requiring milestones to be established in a dependency-aware context. On OnlineBoutique, Qwen3.5-27B drops from 0.75 MC to 0.70 EOC, while GPT-5 drops from 0.64 to 0.55. On TrainTicket, Qwen3.5-Plus and DeepSeek-V4-Flash reach 0.64 and 0.62 MC, but only 0.58 and 0.56 EOC. These gaps indicate that some relevant evidence is observed before its prerequisite evidence has been grounded. Agents therefore exhibit a partly opportunistic search process, while the much larger gap between MC and ECR shows that the more substantial limitation is converting scattered evidence into complete diagnostic closure.

RCA synthesis and diagnostic efficiency are separable capabilities. Strong outcome performance does not necessarily imply complete process grounding. GPT-5 reaches 0.68 JRA on both OnlineBoutique and TrainTicket, but its ECR is only 0.21 and 0.15, respectively. DeepSeek-V4-Flash obtains 0.76 JRA on OnlineBoutique and 0.63 JRA on TrainTicket, while its ECR remains 0.38 and 0.24. These gaps suggest that stronger models can sometimes synthesize a correct RCA from partial but salient signals, but the resulting diagnosis is not always supported by a closed evidence trajectory.

Diagnostic efficiency forms another distinct dimension. EE measures the evidence yield of tool use: the fraction of calls that contribute to admissible, dependency-grounded evidence. On OnlineBoutique, GPT-5 and Qwen3.5-Plus achieve 0.64 and 0.65 EE with only 5.56 and 5.92 steps, respectively, indicating relatively targeted evidence acquisition. In contrast, Qwen3.5-8B uses 16.01 steps but reaches only 0.37 EE, with a high RAR of 0.46. Its longer trajectory therefore does not translate into a comparable amount of grounded evidence. Together, EE and Steps distinguish efficient evidence acquisition from trajectories that are merely long, showing that RCA

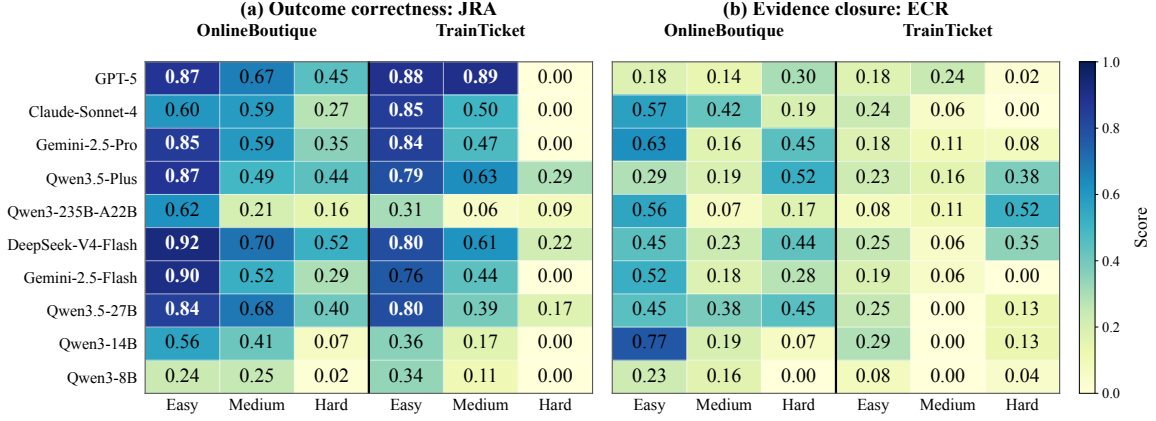


Figure 3: JRA (a) and ECR (b) of LLM agents on OnlineBoutique and TrainTicket across Easy, Medium, and Hard faults.

Table 5: Comparison of experience reuse methods for RCA agents. The best results in each metric are bold.

Model	Method	OnlineBoutique								TrainTicket							
		CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓	CA↑	FA↑	JRA↑	MC↑	EOC↑	ECR↑	EE↑	Steps↓
DeepSeek-V4-Flash	Base	0.88	0.77	0.76	0.74	0.64	0.38	0.62	6.45	0.69	0.71	0.63	0.62	0.56	0.24	0.53	7.68
	ICL	0.90	0.83	0.83	0.83	0.78	0.57	0.64	6.75	0.63	0.61	0.55	0.64	0.58	0.20	0.50	7.08
	SOP	0.92	0.80	0.80	0.81	0.77	0.52	0.69	6.02	0.69	0.70	0.65	0.71	0.64	0.26	0.61	6.75
Gemini-2.5-Flash	Base	0.82	0.67	0.65	0.71	0.67	0.37	0.58	6.49	0.60	0.55	0.53	0.45	0.42	0.12	0.48	4.91
	ICL	0.84	0.74	0.74	0.79	0.76	0.54	0.55	6.88	0.59	0.60	0.58	0.63	0.56	0.14	0.51	6.87
	SOP	0.86	0.74	0.72	0.82	0.76	0.60	0.66	6.29	0.67	0.62	0.62	0.54	0.47	0.09	0.51	5.53
Qwen3.5-27B	Base	0.85	0.71	0.70	0.75	0.70	0.43	0.54	6.87	0.62	0.64	0.58	0.59	0.55	0.18	0.55	7.05
	ICL	0.88	0.79	0.79	0.87	0.83	0.65	0.64	7.35	0.61	0.59	0.54	0.60	0.55	0.20	0.45	6.96
	SOP	0.87	0.78	0.77	0.86	0.81	0.70	0.67	6.70	0.66	0.64	0.60	0.62	0.54	0.17	0.47	6.87

synthesis, process completeness, and diagnostic efficiency capture different agent capabilities.

Redundant tool use mainly appears in weaker models. At the tool interface level, stronger agents generally remain stable in Cloud-OpsBench. GPT-5, Qwen3.5-Plus, Gemini-2.5-Pro and DeepSeek-V4-Flash all show near-zero RAR across the two systems, suggesting that their errors are less about repeatedly issuing identical tool calls and more about evidence grounding, target resolution, or RCA synthesis. In contrast, weaker models exhibit clear redundant exploration. On OnlineBoutique, Qwen3-8B uses 16.01 steps on average and reaches 0.46 RAR, but obtains only 0.19 JRA and 0.16 ECR. Qwen3-14B also follows long trajectories, with 14.58 steps and 0.28 RAR, but reaches only 0.41 JRA. These results show that longer trajectories do not necessarily indicate deeper diagnosis. For weaker agents, additional tool calls often reflect repeated queries or low-value exploration rather than productive verification.

4.4 RQ3: Experience-Augmented RCA Agents

In real cloud operations, engineers rely heavily on accumulated diagnostic experience, including prior incident traces, on-call notes, and runbooks rather than in a zero-shot setting. Therefore, we examine whether reusable operational experience can improve RCA agents. For each root cause, we partitioned the cases into disjoint training and test sets at an approximately 1:1 ratio. The experience pool was constructed exclusively from successful trajectories

collected by GPT-5 on the training set, while the test set was reserved for evaluation. We compare two production-relevant forms of experience reuse. *Trajectory-as-ICL*[5] retrieves three successful trajectories matched by the reported symptom, whereas *Standard Operating Procedure (SOP)*[12, 36] distills recurring diagnostic actions and decision branches into category-level runbook guidance. Because each broad fault category contains multiple root causes, neither method uses the target root-cause label for retrieval. We select Gemini-2.5-Flash, Qwen3.5-27B, and DeepSeek-V4-Flash for this ablation because they combine competitive base performance with relatively low inference cost and latency.

Experience reuse improves RCA, but its gains depend on retrieval granularity. On OnlineBoutique, both methods consistently improve all three models. Averaged across models, JRA increases from 0.70 in the base setting to 0.79 with ICL and 0.76 with SOP, while ECR rises from 0.39 to 0.59 and 0.61, respectively. On TrainTicket, the gains are less uniform. SOP raises average JRA from 0.58 to 0.62, whereas ICL reaches 0.56. Both methods improve partial process alignment: average MC increases from 0.55 to 0.62 with either method, and EOC increases from 0.51 to 0.56 with ICL and 0.55 with SOP. However, full evidence closure remains nearly unchanged, with ECR values of 0.18, 0.18, and 0.17 for Base, ICL, and SOP.

Taken together, these results show that successful historical experience can improve both RCA outcomes and process alignment, with the most consistent gains on OnlineBoutique. The less uniform TrainTicket results can be understood through retrieval granularity. Since the root cause is unknown at inference time, we retrieve experience using the observable symptom rather than an oracle root-cause label. ICL receives only three demonstrations, and the same symptom may correspond to multiple root causes. Consequently, the retrieved cases may not match the mechanism in the current incident. SOP instead aggregates recurring diagnostic branches across the root causes within a fault category, providing broader but less case-specific guidance. Operational experience is therefore a useful augmentation source, while retrieval quality and knowledge abstraction determine how consistently its benefits transfer across systems.

4.5 Failure Modes of Cloud RCA Agents

To understand agent errors beyond aggregate scores, we first collect failure cases with JRA = 0 from the ten evaluated models. From this failure pool, we qualitatively inspect a purposive sample of trajectories covering both benchmark systems, all eight fault categories, and models with different capability levels. This exploratory analysis characterizes representative and potentially overlapping behaviors rather than estimating their prevalence or defining an exhaustive taxonomy. We organize the observed failures around four broad forms of diagnostic breakdown.

(1) Evidence-grounding failures. Agents sometimes retrieve useful evidence but interpret it incorrectly or assign it insufficient weight. For example, startup events may already report manifest unknown, 403 Forbidden, or a missing secret, yet the agent continues checking peripheral resources. Agents can also anchor on salient downstream symptoms, such as frontend 503 responses, high latency, or a prominent CPU alert, and mistake the noisiest component for the causal source. Conversely, missing objects and telemetry can be overlooked: an empty endpoint set, an absent pod, or the lack of logs from a failed component is sometimes treated as no evidence rather than as a diagnostic signal. Such misinterpretations can place the investigation in the wrong fault family, for example by explaining a code defect as OOM, network delay, or service misconfiguration.

(2) Search-control failures. Agents do not always adapt their investigation to the evidence already established. Some commit to a plausible hypothesis after inspecting one abnormal pod or service and stop before checking competing explanations. Others fail to redirect the search when an initial path is weakened; for instance, after confirming normal service connectivity, they may not move to gateway routing or configuration checks. The opposite behavior also occurs: even after observing decisive evidence such as replicas=0, an inactive kube-proxy, or an explicit database access error, an agent may continue issuing low-value or repeated queries. These trajectories reflect weak control over hypothesis revision, diagnostic branching, and stopping.

(3) Target-resolution failures. Relevant evidence does not necessarily lead agents to the correct diagnostic target. In TrainTicket, database errors observed by application services may be attributed to those services instead of the shared tsdb-mysql dependency;

similarly, gateway routing faults may be assigned to affected business services. Agents also confuse closely related mechanisms, such as image-reference and registry-DNS failures, probe port and protocol errors, or adjacent code-defect subtypes. Codedefect cases further require connecting service-level alerts, HTTP errors, and abnormal metrics to source-level behavior. Agents may inspect relevant telemetry or code artifacts but still explain a busy loop, artificial delay, or excessive file operation as resource saturation or a performance fault.

(4) Tool-grounding failures. Failures also arise at the interface between reasoning and execution. Invalid parameters, an incorrect object or scope, or an action inconsistent with the intended check can prevent the required evidence from being obtained. More subtly, a syntactically valid but mis-specified query can create misleading evidence. For example, probing service connectivity with the wrong port produces a genuine connection failure that the agent may then treat as a production symptom and use to support an incorrect diagnosis.

Our process-oriented analysis reveals that agent failures often develop before the final prediction, through breakdowns in evidence-grounded investigation and reasoning. These observations show that improving current RCA agents requires not only better final predictions, but also more reliable and interpretable diagnostic trajectories.

5 Discussion

5.1 Implications and Research Utility

Cloud-OpsBench is an evaluation infrastructure for interactive and evidence-grounded cloud RCA. Its contribution extends beyond a collection of cloud fault cases by making both the final diagnosis and its supporting investigation controlled and measurable.

Controlled experiments on cloud diagnosis. By replaying the same runtime-verified incident, Cloud-OpsBench allows researchers to compare RCA agents or vary one agent component while holding the available fault evidence fixed. For example, planners, memory mechanisms, topology and knowledge retrieval, and tool-selection policies can be compared using the same user query and incident state. Differences in diagnostic correctness, milestone coverage, evidence-order consistency, and investigation efficiency can therefore be attributed more directly to agent design rather than to a different realization of the fault. Unlike static telemetry inputs, the replay interfaces preserve the agent's decisions about which diagnostic tool, component, and evidence source to inspect.

Process-level analysis of RCA agents. The diagnostic evidence graph aligns different tool-mediated investigations according to the causal facts they establish. For a failure that propagates across services or platform resources, it can show whether an agent observes the reported manifestation but fails to traverse the relevant dependency, reaches the faulty component without confirming its mechanism, or concludes before acquiring sufficient evidence. Alternative evidence bundles and partial-order dependencies allow these failures to be analyzed without requiring agents to reproduce a canonical command sequence. This enables targeted studies of whether topology information improves propagation tracing, retrieved operational knowledge improves mechanism confirmation,

or memory and planning reduce repeated or causally premature investigation.

Reusable diagnostic experience. Trajectories produced on Cloud-OpsBench can be aligned to the same milestones even when agents follow different investigation paths. This alignment makes successful and failed strategies comparable across models, fault types, and application and platform layers, providing a basis for studying how diagnostic experience transfers across incidents. Although this work focuses on evaluation, the resulting trajectories can also support future research on experience retrieval, diagnostic skill distillation, and process-aware training for cloud RCA agents.

5.2 Future Research Opportunities

State-adaptive evidence acquisition. Current RCA agents often rely on generic investigation routines that do not adapt sufficiently to the evidence already established. Future work can study policies that select the next tool and target according to the grounded milestones, unresolved diagnostic alternatives, and expected information gain. An important question is whether topology-aware planning and uncertainty-aware stopping can improve evidential completeness while avoiding premature conclusions and unnecessary inspection.

Transferable diagnostic skills. Cloud incidents differ in their components and topologies, but often require recurring diagnostic operations such as dependency traversal, failure-boundary localization, and mechanism confirmation. Aligning trajectories through diagnostic milestones provides a basis for learning or retrieving skills that are invariant to component identifiers and exact command syntax. Future studies should examine whether experience acquired from one workload or fault type improves investigation of unseen incidents without relying on root-cause labels or memorized case-specific paths.

RCA under incomplete and interacting evidence. The current benchmark focuses on individually verified faults whose required evidence is captured in the verified incident state or its associated case-specific incident interval. Operational incidents may instead involve concurrent faults, delayed effects, changing system states, and missing or contradictory observations. Extending the construction pipeline to such settings would enable research on maintaining competing hypotheses, distinguishing root causes from propagated symptoms, and expressing uncertainty when the available evidence does not support a unique diagnosis. Such extensions should preserve matched evaluation conditions and auditable evidence requirements rather than introducing complexity at the expense of comparability.

5.3 Threats to Validity

External Validity. Cloud-OpsBench focuses on Kubernetes-based cloud-native systems, which may limit generalization to other architectures and operational environments. We mitigate this threat by covering 57 fault types drawn from multiple sources, two workloads with different service scales, both application and platform layers, and retaining only runtime-verified cases. Nevertheless, broader validation requires additional workloads, deployment configurations, and real-world incidents. Each benchmark release also reflects the Kubernetes version used during fault realization and snapshot capture. Most supported diagnostic interfaces rely on stable API

resources and long-standing troubleshooting operations governed by Kubernetes compatibility and deprecation policies [23], which reduces drift under routine upgrades. When a Kubernetes update changes a fault manifestation or diagnostic interface, the construction pipeline can re-execute the affected cases and recapture their snapshots as a revalidated, versioned release.

Internal Validity. Model comparisons may be affected by the agent scaffold, prompting choices, inference nondeterminism, and differences in prior exposure to Kubernetes knowledge. We mitigate these factors by using the same ReAct framework, prompt structure, tool schemas, step budget, temperature setting, and snapshot evidence across models. Because process credit requires case-specific observations acquired through interaction, prior domain knowledge alone cannot obtain a complete process score.

Construct Validity. Cloud-OpsBench evaluates diagnosis from evidence preserved in the verified incident state and its associated case-specific incident interval rather than from arbitrary system history or subsequent live state transitions. We mitigate this limitation by covering the principal evidence channels used in cloud-native diagnosis and retaining only cases whose required milestones are observable and replay-validated. Faults whose diagnosis requires long-term historical baselines, evidence preceding baseline confirmation, or observations produced only after verification are outside the current benchmark scope.

6 Related Work

Evolution of RCA. Traditional AIOps RCA formulates diagnosis as discriminative localization over metrics, logs, and traces, ranking faulty components with causal or graph models [16, 25, 26, 29, 30, 50–52, 59]. LLM-based systems then treat RCA as generative analysis over a provided incident context, often retrieving operational documents to produce explanations [1, 9, 13, 28, 46]. Recent agentic workflows go further: they select diagnostic tools, interpret returned observations, and revise hypotheses through multi-step interaction with cloud and related operational environments [15, 34, 36, 43, 44, 48, 57, 58]. This shift makes interactive evidence acquisition central to RCA and motivates a new class of benchmarks that can evaluate not only final root-cause labels, but also tool-mediated investigation under controlled, comparable incident conditions.

Benchmarking in AIOps. Existing AIOps benchmarks only partially meet this need. Static telemetry benchmarks provide deterministic offline artifacts for detection or localization, but agents cannot investigate through operational interfaces [2, 3, 10, 25, 37, 40, 52, 54, 61]. OpenRCA allows code-based queries over supplied files, which improves inspectability without exposing system-facing SRE tools [47]. Live-environment benchmarks restore standard tool use on real services, but fault manifestations can vary across runs, which complicates controlled comparison and fine-grained process scoring [8, 18, 42]. Across both paradigms, evaluation remains largely outcome-oriented. As summarized in Table 1, Cloud-OpsBench targets the missing combination: snapshot-backed replay preserves system-facing investigation under matched incident observations, and diagnostic evidence graphs score whether required facts are established in an admissible order from tool observations, rather than only whether the final root-cause label is correct or whether the agent reproduces a single expert command sequence.

7 Conclusion

This paper presents Cloud-OpsBench, an evaluation infrastructure for interactive and evidence-grounded cloud RCA. It combines runtime-verified fault realization with snapshot-backed replay, allowing agents to investigate matched incident observations through standard diagnostic interfaces while independently selecting their tools and targets. Each case pairs an outcome label with a diagnostic evidence graph, enabling evaluation of root-cause correctness, milestone coverage, evidence-order consistency, and investigation efficiency without prescribing a canonical command sequence. With 754 cases spanning 57 fault types across application and Kubernetes platform layers, Cloud-OpsBench provides a controlled basis for comparing RCA agents and analyzing where their investigations fail in evidence acquisition, dependency traversal, mechanism confirmation, and final diagnosis.

References

- [1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In *ICSE 2023*. IEEE, 1737–1749. doi:10.1109/ICSE48619.2023.00149
- [2] AIOPS.CN. 2020. AIOPS 2020 challenges data. <https://github.com/NetManAIops/AIOps-Challenge-2020-Data>.
- [3] AIOPS.CN. 2021. AIOPS 2021 challenges data. <https://www.aiops.cn/gitlab/aiops-nankai/data/trace/aiops2021>.
- [4] Marco Barletta, Marcello Cinque, Catello Di Martino, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2024. Mutiny! How Does Kubernetes Fail, and What Can We Do About It?. In *DSN 2024*. IEEE, 1–14. doi:10.1109/DSN58291.2024.00016
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [6] Chaosblade. 2026. Chaosblade. <https://chaosblade.io/>.
- [7] Junjie Chen, Shu Zhang, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Yu Kang, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2020. How Incidental are the Incidents? Characterizing and Prioritizing Incidents for Large-Scale Online Service Systems. In *ASE 2020*. IEEE, 373–384. doi:10.1145/3324884.3416624
- [8] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOPS Lab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *MLSys 2025*. <https://openreview.net/forum?id=3EXBLwGxtq>
- [9] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *EuroSys 2024*. ACM, 674–688. doi:10.1145/3627703.3629553
- [10] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Chenyu Zhao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, Yuzhe Cai, Weibin Meng, Yongqian Sun, and Dan Pei. 2025. LogEval: A comprehensive benchmark suite for LLMs in log analysis. *Empir. Softw. Eng.* 30, 6 (2025), 173. doi:10.1007/S10664-025-10701-6
- [11] Rob Ewaschuk. 2016. Monitoring Distributed Systems. In *Site Reliability Engineering: How Google Runs Production Systems*, Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). O'Reilly Media, Chapter 6. <https://sre.google/sre-book/monitoring-distributed-systems/>
- [12] Yufei Gao, Zhengong Cai, and Bowei Yang. 2026. RCAFlow: A Workflow-Informed Hierarchical Planning Multi-Agent System for Root Cause Analysis. In *AAAI 2026*, Vol. 40. 300–308. doi:10.1609/aaai.v40i1.36991
- [13] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjali Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-LifeCycle Learning for Cloud Incident Management using LLMs. In *FSE Companion 2024*. ACM, 417–428. doi:10.1145/3663529.3663861
- [14] Google. 2026. Online Boutique: Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC. <https://github.com/GoogleCloudPlatform/microservices-demo>.
- [15] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model Based Multi-agents: A Survey of Progress and Challenges. In *IJCAI 2024*. ijcai.org, 8048–8057. <https://www.ijcai.org/proceedings/2024/890>
- [16] Jin Huang, Pengfei Chen, Guangba Yu, Yilun Wang, Haiyu Huang, and Zilong He. 2024. FaaSACA: Full Lifecycle Root Cause Analysis for Serverless Applications. In *ISSRE 2024*. IEEE, 415–426. doi:10.1109/ISSRE62328.2024.00047
- [17] Istio. 2026. Service Mesh. Simplified. <https://istio.io/>.
- [18] Saurabh Jha, Rohan R. Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O. Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Jae-wook Ahn, Debanjana Kar, Amit M. Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Schwartz, Daby Sow, Nicholas C. Fuller, and Ruchir Puri. 2025. IT-Bench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *ICML 2025*. OpenReview.net. <https://openreview.net/forum?id=jP59rz1bZk>
- [19] Chris Jones. 2016. Effective Troubleshooting. In *Site Reliability Engineering: How Google Runs Production Systems*, Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). O'Reilly Media, Chapter 12. <https://sre.google/sre-book/effective-troubleshooting/>
- [20] k8sdock. 2025. Kubernetes Documentation. Retrieved June 10, 2026 from <https://kubernetes.io/docs/concepts/>
- [21] NVJK Kartik, Garvit Sapra, Rishav Hada, and Nikhil Pareek. 2025. Agent-Compass: Towards Reliable Evaluation of Agentic Workflows in Production. arXiv:2509.14647 [cs.AI] <https://arxiv.org/abs/2509.14647>
- [22] KodeKloud. 2025. KodeKloud Notes. Retrieved June 10, 2026 from <https://notes.kodekloud.com/>
- [23] Kubernetes Authors. 2024. Kubernetes Deprecation Policy. Retrieved Jul. 13, 2026 from <https://kubernetes.io/docs/reference/using-api/deprecation-policy/>
- [24] LearnKube.CN. 2025. LearnKUBE. <https://learnkubernetes.com/troubleshooting-deployments>.
- [25] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R Lyu. 2023. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.
- [26] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. 2023. SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. *IEEE Trans. Dependable Secur. Comput.* 20, 4 (2023), 2762–2780. doi:10.1109/TDSC.2022.3162857
- [27] Xiaoyun Li, Guangba Yu, Pengfei Chen, Hongyang Chen, and Zhekang Chen. 2022. Going through the Life Cycle of Faults in Clouds: Guidelines on Fault Handling. In *ISSRE 2022*. IEEE, 121–132. doi:10.1109/ISSRE55969.2022.00022
- [28] Yichen Li, Yulun Wu, Jinyang Liu, Zhihan Jiang, Zhuangbin Chen, Guangba Yu, and Michael R. Lyu. 2025. COCA: Generative Root Cause Analysis for Distributed Systems with Code Knowledge. In *ICSE 2025*. IEEE, 1346–1358. doi:10.1109/ICSE55347.2025.00234
- [29] Yufeng Li, Guangba Yu, Pengfei Chen, Chuanfu Zhang, and Zibin Zheng. 2022. MicroSketch: Lightweight and Adaptive Sketch Based Performance Issue Detection Localization in Microservice Systems. In *ICSOC 2022*. Springer-Verlag, Berlin, Heidelberg, 219–236. doi:10.1007/978-3-031-20984-0_15
- [30] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchi Zhang, Kaixin Sui, Yanhua Wang, Xu Du, Guoqiang Duan, and Dan Pei. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *ESEC/FSE 2022*. ACM, 996–1008. doi:10.1145/3540250.3549092
- [31] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=v8L0pN6EOi>
- [32] Locust. 2026. An open source load testing tool. <https://locust.io>.
- [33] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems* 37 (2024), 74325–74362.
- [34] Jiayi Mao, Liqun Li, Yanjie Gao, Zegang Peng, Shilin He, Chaoyun Zhang, Si Qin, Samia Khalid, Qingwei Lin, Saravan Rajmohan, Sitaram Lanka, and Dongmei Zhang. 2026. StepFly: Agentic Troubleshooting Guide Automation for Incident Diagnosis. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE136 (June 2026), 23 pages. doi:10.1145/3808143
- [35] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. 2024. WebCanvas: Benchmarking Web Agents in Online Environments. arXiv:2406.12373 [cs.CL] <https://arxiv.org/abs/2406.12373>
- [36] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, and Dan Pei. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *WWW 2025*. Association for Computing Machinery, 422–431. doi:10.1145/3701716.3715225

- [37] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *WWW 2025*. ACM, 777–780. doi:10.1145/3701716.3715290
- [38] Prometheus. 2026. Open source metrics and monitoring for your systems and services. <https://prometheus.io/>.
- [39] Pydantic. 2026. A widely used data validation library for Python. <https://docs.pydantic.dev/latest/>.
- [40] Haotian Si, Jianhui Li, Changhua Pei, Hang Cui, Jingwen Yang, Yongqian Sun, Shenglin Zhang, Jingjing Li, Haiming Zhang, Jing Han, Dan Pei, and Gaogang Xie. 2024. TimeSeriesBench: An Industrial-Grade Benchmark for Time Series Anomaly Detection Models. In *ISSRE 2024*. IEEE, 61–72. doi:10.1109/ISSRE62328.2024.00017
- [41] Stackoverflow. 2025. Stack Overflow. Retrieved June 10, 2026 from <https://stackoverflow.com/questions>
- [42] Yongqian Sun, Jiaju Wang, Zhengdan Li, Xiaohui Nie, Minghua Ma, Shenglin Zhang, Yuhe Ji, Lu Zhang, Wen Long, Hengmao Chen, Yongnan Luo, and Dan Pei. 2025. AIOpsArena: Scenario-Oriented Evaluation and Leaderboard for AIOps Algorithms in Microservices. In *SANER 2025*. IEEE, 809–813. doi:10.1109/SANER64311.2025.00082
- [43] Yifang Tian, Yaming Liu, Zichun Chong, Zihang Huang, and Hans-Arno Jacobsen. 2025. GALA: Can Graph-Augmented Large Language Model Agentic Workflows Elevate Root Cause Analysis? *CoRR* abs/2508.12472 (2025). arXiv:2508.12472 doi:10.48550/ARXIV.2508.12472
- [44] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *CIKM 2024*. ACM, 4966–4974. doi:10.1145/3627673.3680016
- [45] Zirui Wang, Guangba Yu, and Michael R. Lyu. 2026. AI-NativeBench: An Open-Source White-Box Agentic Benchmark Suite for AI-Native Systems. *ACM Trans. Softw. Eng. Methodol.* (July 2026). doi:10.1145/3821576
- [46] Yong Xiang, Charley Peter Chen, Liyi Zeng, Wei Yin, Xin Liu, Hu Li, and Wei Xu. 2025. Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM. *CoRR* abs/2506.02490 (2025). arXiv:2506.02490 doi:10.48550/ARXIV.2506.02490
- [47] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *ICLR 2025*. OpenReview.net. <https://openreview.net/forum?id=M4qNlZQYpd>
- [48] Yitao Yang, Yangtao Deng, Yifan Xiong, Baochun Li, Hong Xu, and Peng Cheng. 2025. AidAI: Automated Incident Diagnosis for AI Workloads in the Cloud. *CoRR* abs/2506.01481 (2025). arXiv:2506.01481 doi:10.48550/ARXIV.2506.01481
- [49] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- [50] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Ximmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *WWW 2021*. ACM, 3087–3098. doi:10.1145/3442381.3449905
- [51] Guangba Yu, Pengfei Chen, Zilong He, Qiuyu Yan, Yu Luo, Fangyuan Li, and Zibin Zheng. 2024. ChangeRCA: Finding Root Causes from Software Changes in Large Online Systems. *Proc. ACM Softw. Eng.* 1, FSE (2024), 24–46. doi:10.1145/3643728
- [52] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *ESEC/FSE 2023*. ACM, 553–565. doi:10.1145/3611643.3616249
- [53] Guangba Yu, Pengfei Chen, and Zibin Zheng. 2022. Microscaler: Cost-effective Scaling for Microservice Applications in the Cloud with an Online Learning Approach. *IEEE TCC* 10, 2 (2022), 1100–1116. doi:10.1109/TCC.2020.2985352
- [54] Chaoli Zhang, Yingying Zhang, Lanshu Peng, Qingsong Wen, Yiyuan Yang, Chongjiong Fan, Minqi Jiang, Lunting Fan, and Liang Sun. 2024. Advancing Multivariate Time Series Anomaly Detection: A Comprehensive Benchmark with Real-World Data from Alibaba Cloud. In *CIKM 2024*. ACM, 5410–5414. doi:10.1145/3627673.3679128
- [55] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Minghua He, Leyi Pan, Zhaoyang Liu, Bolin Ding, and Ying Li. 2025. MicroRemed: Benchmarking LLMs in Microservices Remediation. arXiv:2511.01166 [cs.CL] <https://arxiv.org/abs/2511.01166>
- [56] Renyi Zhong, Yichen Li, Guangba Yu, Wenwei Gu, Jinxi Kuang, Yintong Huo, and Michael R. Lyu. 2025. Larger Is Not Always Better: Exploring Small Open-source Language Models in Logging Statement Generation. *ACM Trans. Softw. Eng. Methodol.* (2025). doi:10.1145/3773287
- [57] Yuedong Zhong, Guangba Yu, Yujie Huang, QunChao Fu, Rui Ren, Cong Feng, Yongqiang Yang, and Michael Lyu. 2026. LLMGuard: Multi-Agent Fault Diagnosis for Reliable Language-Model-as-a-Service. In *2026 56th Annual IEEE International Conference on Dependable Systems and Networks (DSN)*. 15–23. doi:10.1109/DSN69566.2026.00018
- [58] Wei Zhou, Peng Sun, Xuanhe Zhou, Qianglei Zang, Ji Xu, Tieying Zhang, Guoliang Li, and Fan Wu. 2025. DBAIOps: A Reasoning LLM-Enhanced Database Operation and Maintenance System using Knowledge Graphs. arXiv:2508.01136 [cs.DB] <https://arxiv.org/abs/2508.01136>
- [59] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Software Eng.* 47, 2 (2021), 243–260. doi:10.1109/TSE.2018.2887384
- [60] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *ICSE 2018 Poster*. ACM, 323–324. doi:10.1145/3183440.3194991
- [61] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. 2023. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In *ISSRE 2023*. IEEE, 355–366. doi:10.1109/ISSRE59848.2023.00071